



4. Deep Learning

Subject : Fundamental of AI (BE02000041)

Credit: 2 (L: 2; T: 0; P: 0)

ESE(E)-70, PA/CA-30; PA stands for Progressive Assessment and CA stands for Credit)

Faculty:

Ms. Pooja D. Pariyani(CO), Assistant Professor, pooja.pariyani@ckpcet.ac.in

Prof. Chaitali V. Sheth(CO), Assistant Professor, chaitali.sheth@ckpcet.ac.in

Prof. Nidhi Periwal(CO), Assistant Professor, nidhi.periwal@ckpcet.ac.in

Prof. Shivani V. Vora(IT), Assistant Professor, shivani.vora@ckpcet.ac.in

Dr. Soumita D. Bid, Assistant Professor, soumita.bid@ckpcet.ac.in

Dr. Tandra J. Banerjee(CI), Assistant Professor, tandara.banerjee@ckpcet.ac.in

Dr. Mita C. Paunwala(ECC), Associate Professor, mita.paunwala@ckpcet.ac.in

Dr. Ami T. Choksi(CO & IT), Associate Professor(CO) & HOD(IT), ami.choksi@ckpcet.ac.in

Navyug Vidyabhavan Trust

C.K. Pithawala College of Engineering and Technology, Surat, Gujarat State, India. Website: www.ckpcet.ac.in

Syllabus:

<https://drive.google.com/drive/folders/1IVZrDRTzvif6lGzCvjhdHhV-U6tWKpAb?usp=sharing>



Learning Material(Books:)

1. Artificial Intelligence: Concepts and Applications by Lavika Goel, Latest Edition, Wiley.
2. Artificial Intelligence by Kevin Knight, Elaine Rich, Shivashankar B. Nair, Latest Edition, McGraw Hill.
3. Understanding Artificial Intelligence: Fundamentals and Applications by Albert Chun-Chen Liu, Oscar Ming Kin Law, Iain Law, Latest Edition Wiley-IEEE Press.
4. Fundamentals of Artificial Intelligence by K.R. Chowdhary, Latest Edition, Springer.
5. Artificial Intelligence A Modern Approach by Stuart J. Russell and Peter Norvig, Latest Edition, Pearson.

E-books downloaded on my drive.

<https://drive.google.com/drive/folders/1cnsJ5Au3QBewW9hrVYli2AK6uIfnYPkY?usp=sharing>



Module 4: Deep Learning [6 Lectures][20%]

- Concept of Deep Learning
- Introduction to Neural Networks.
- Types of Deep Learning models.
- Deep learning applications.

CO4: Understand the Deep Learning concepts and its applications for AI.

Motivation:



"two young girls are playing with lego toy."



"boy is doing backflip on wakeboard."



"construction worker in orange safety vest is working on road."

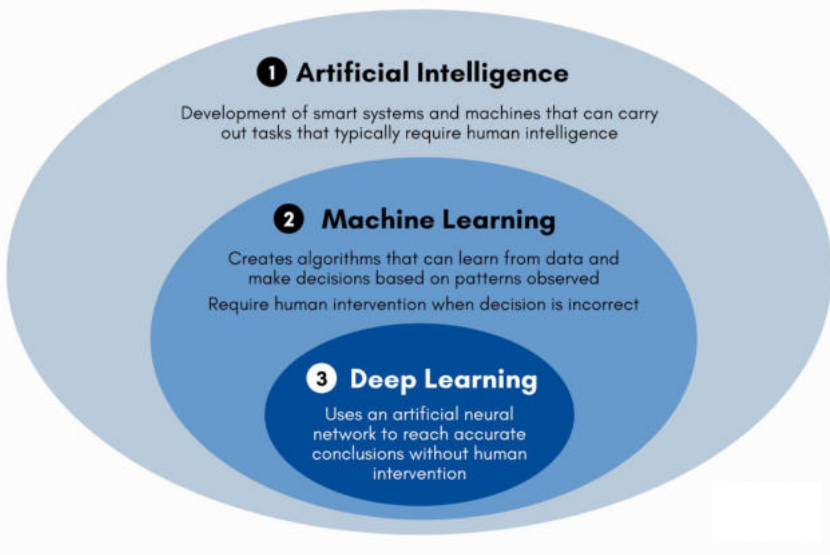


"man in black shirt is playing guitar."



Concept of Deep Learning:

ARTIFICIAL INTELLIGENCE VS MACHINE LEARNING VS DEEP LEARNING



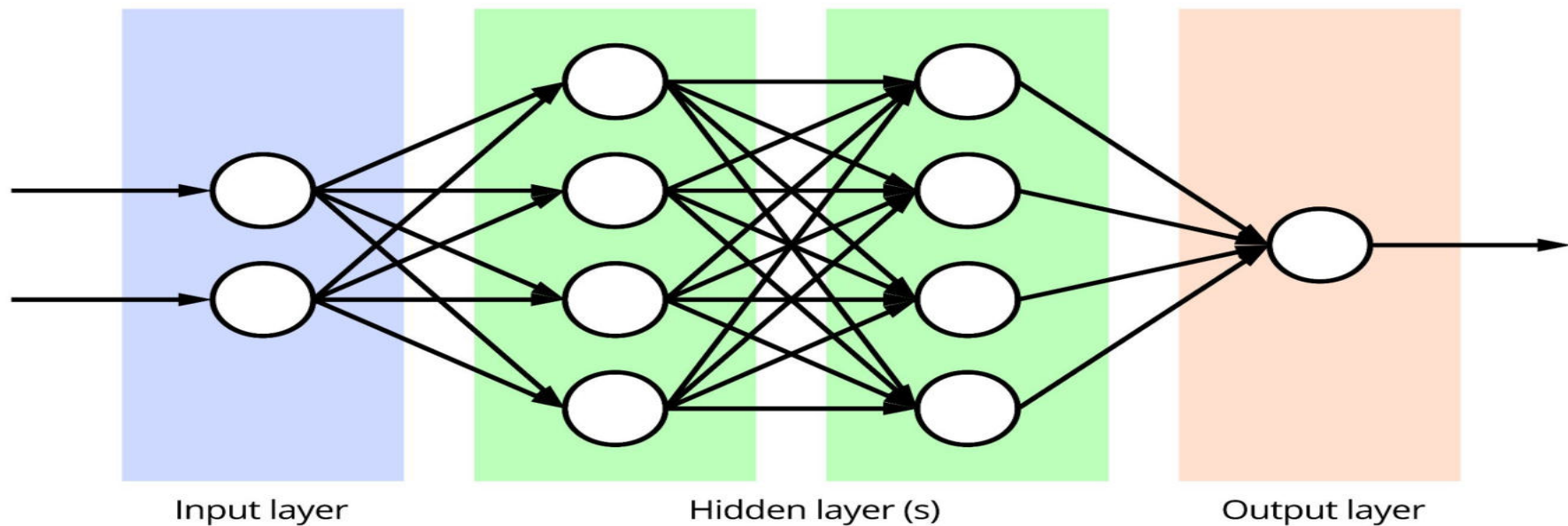
Definition 1: Deep learning is a branch of machine learning based on Artificial Neural Network (ANN) that teaches computers to perform tasks by learning from examples, much like humans do.

An ANN uses layers of interconnected nodes called neurons that works together to process and learn from input data.

Just like ML, DL also uses large amount of data to learn.



Deep Learning architecture



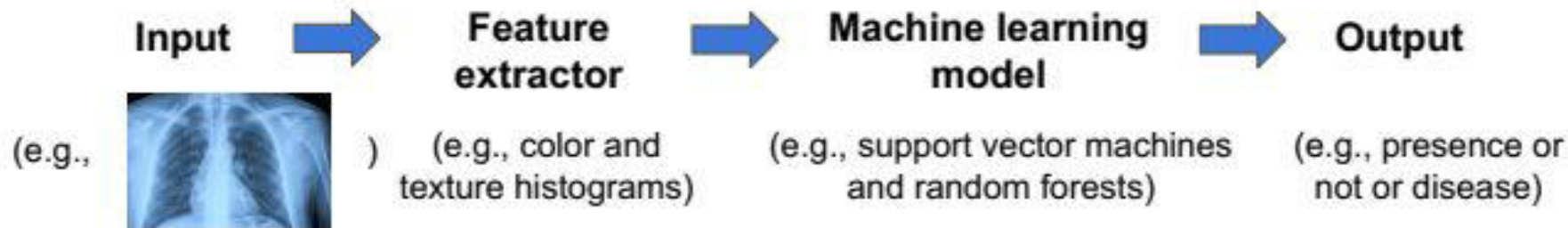


Deep Learning architecture

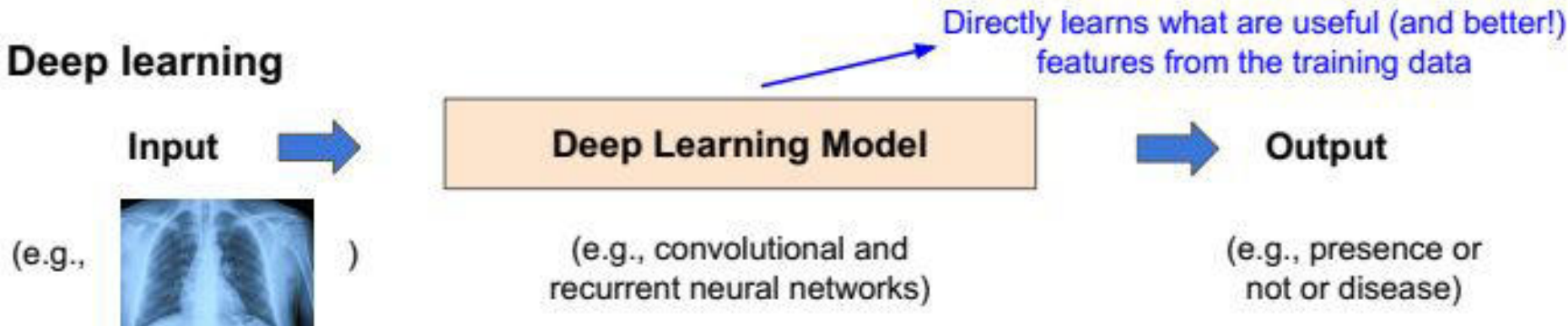
- **Layers:** input, hidden,output (data enters, perform calculation and transformation, produce final result)
- **Neural Network:** process the data and passes it to next layer to learn complex pattern
- **Activation Function:** decide whether a neuron should be activated or not and introduce non linearity in model

Difference: Machine Learning vs Deep Learning

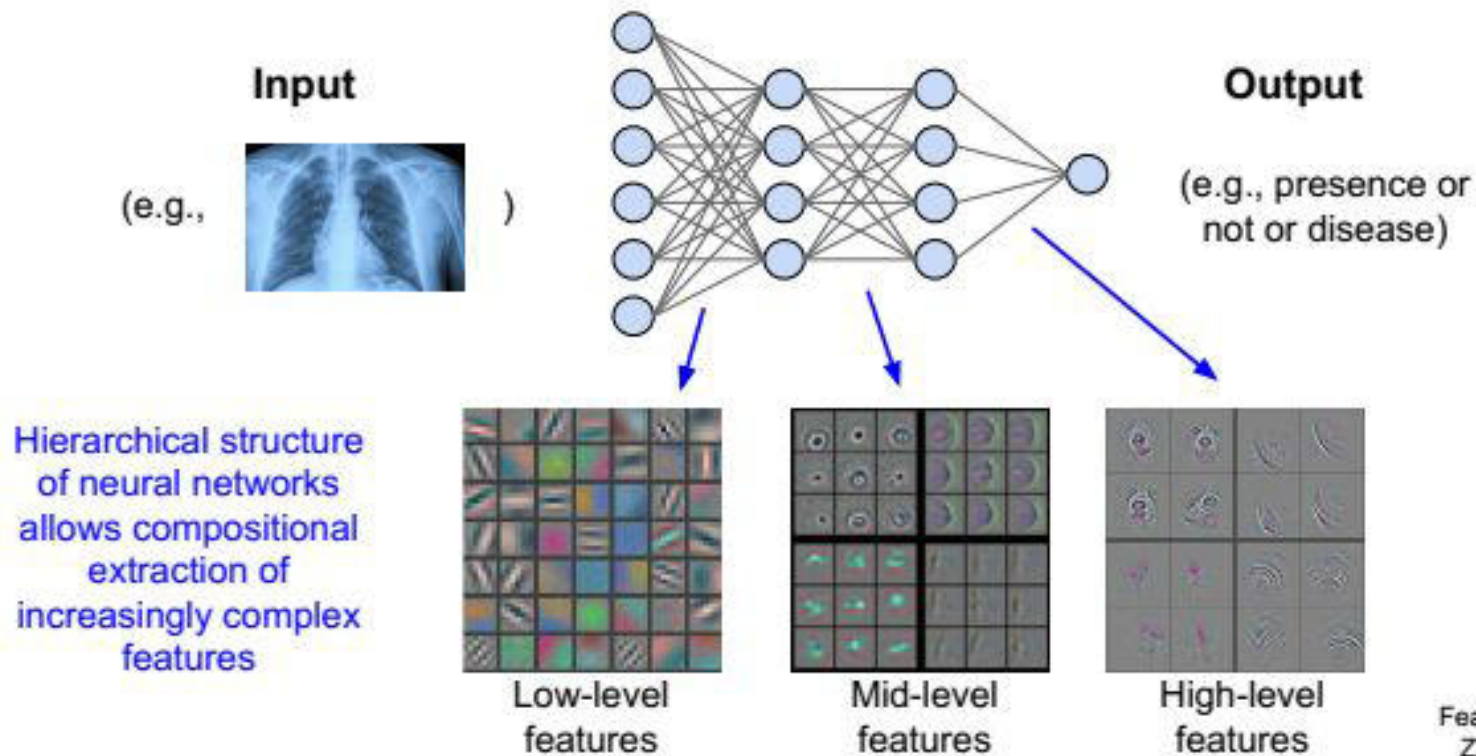
Traditional machine learning



Deep learning



Extraction of Features in Deep Learning:

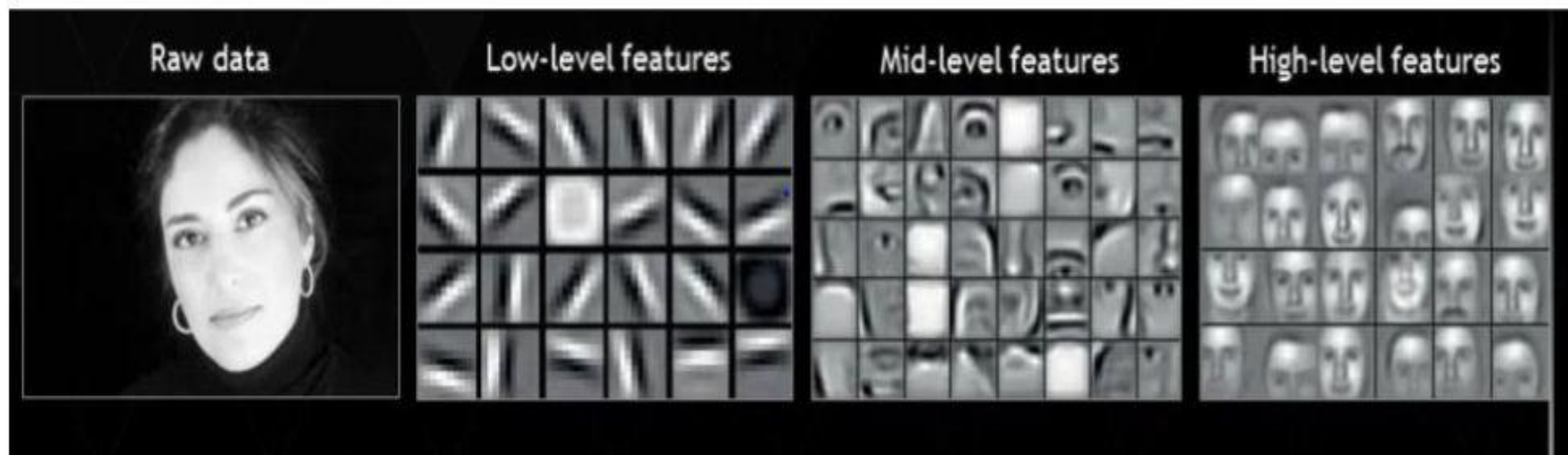


Example: Feature Engineering in deep learning

Low Level Features: Edges, Lines etc.

Mid Level Features: Parts of faces

High Level Features: High level presentation of face



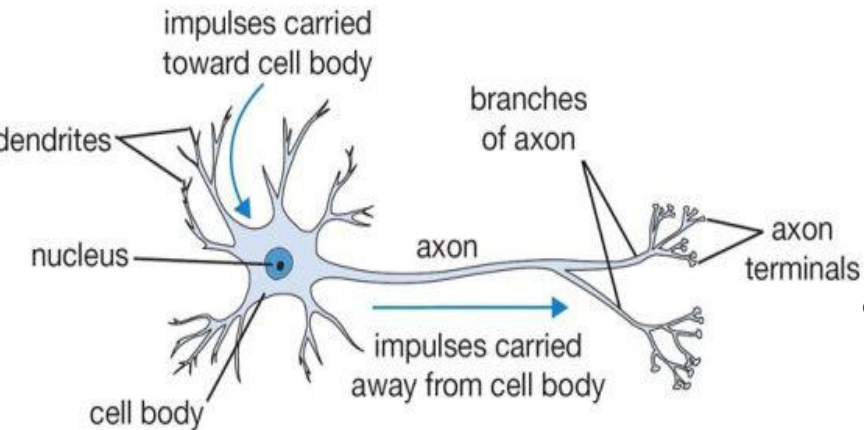


Difference: Machine Learning vs Deep Learning

Feature	Machine Learning	Deep Learning
Data Required	Can work with smaller datasets	Requires large amounts of data
Feature Extraction	Often requires manual feature engineering	Feature extraction is often automated by the model
Model Complexity	Simpler models (e.g., linear regression, SVMs)	Complex models (e.g., neural networks with many layers)
Computational Resources	Less computationally intensive	Highly computationally intensive
Training Time	Generally faster training	Can take a long time to train
Interpretability	Easier to interpret results	Often difficult to understand why a model makes a decision
Applications	Classification, regression, clustering, etc.	Image recognition, natural language processing, speech recognition, etc.
Human Input	More human input often required for feature engineering and model selection	Less human input required for feature engineering, model selection.
Performance on Complex Tasks	May struggle with very complex, unstructured data	Excels at complex tasks with unstructured data
Hardware Requirements	Can often run on standard hardware	Often requires specialized hardware (GPUs)
Learning Method	Can use various algorithms (e.g., decision trees, random forests)	Primarily uses artificial neural networks

Working Principle of Neuron

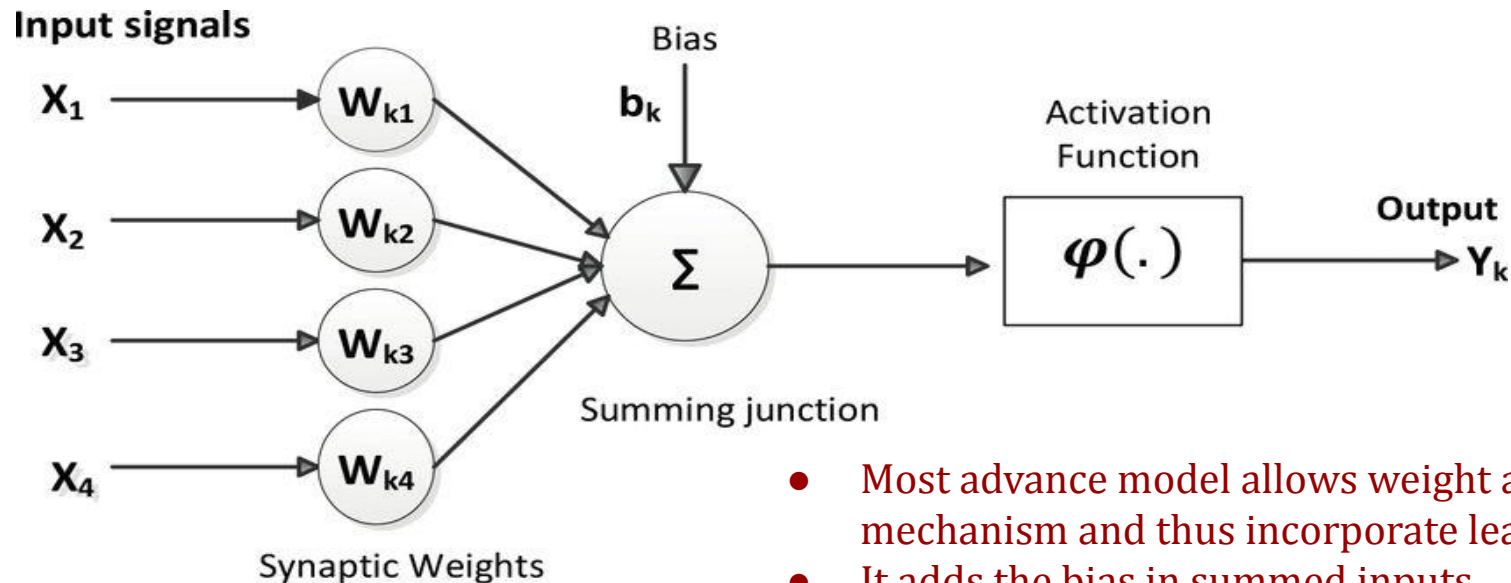
Biological Neuron



- Biological Neuron Structure: Each biological neuron has three main parts: the cell body, dendrites, and an axon. Dendrites receive incoming signals, and the axon sends signals out. When the cell body sums up enough input, it generates an electrical impulse.
- Synaptic Connections: Biological neurons connect at synapses. Stronger connections make it easier for signals to move from neuron to neuron.



Artificial neuron (McCulloch-Pitts neuron_1943)

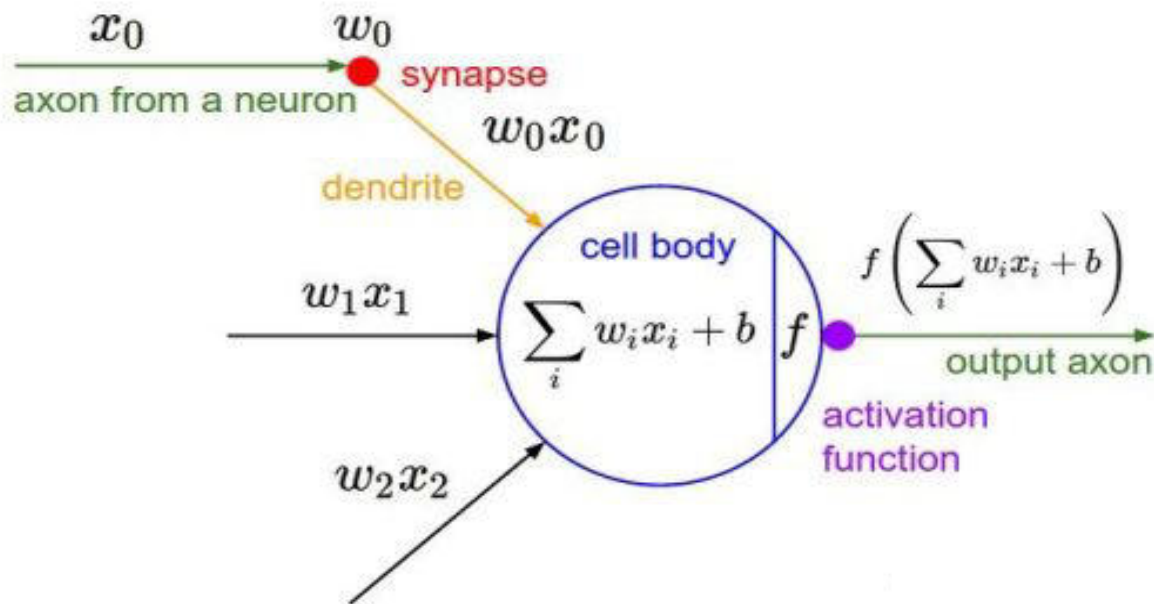


- Most advance model allows weight adjustment mechanism and thus incorporate learning.
- It adds the bias in summed inputs
- Uses various non linear activation function

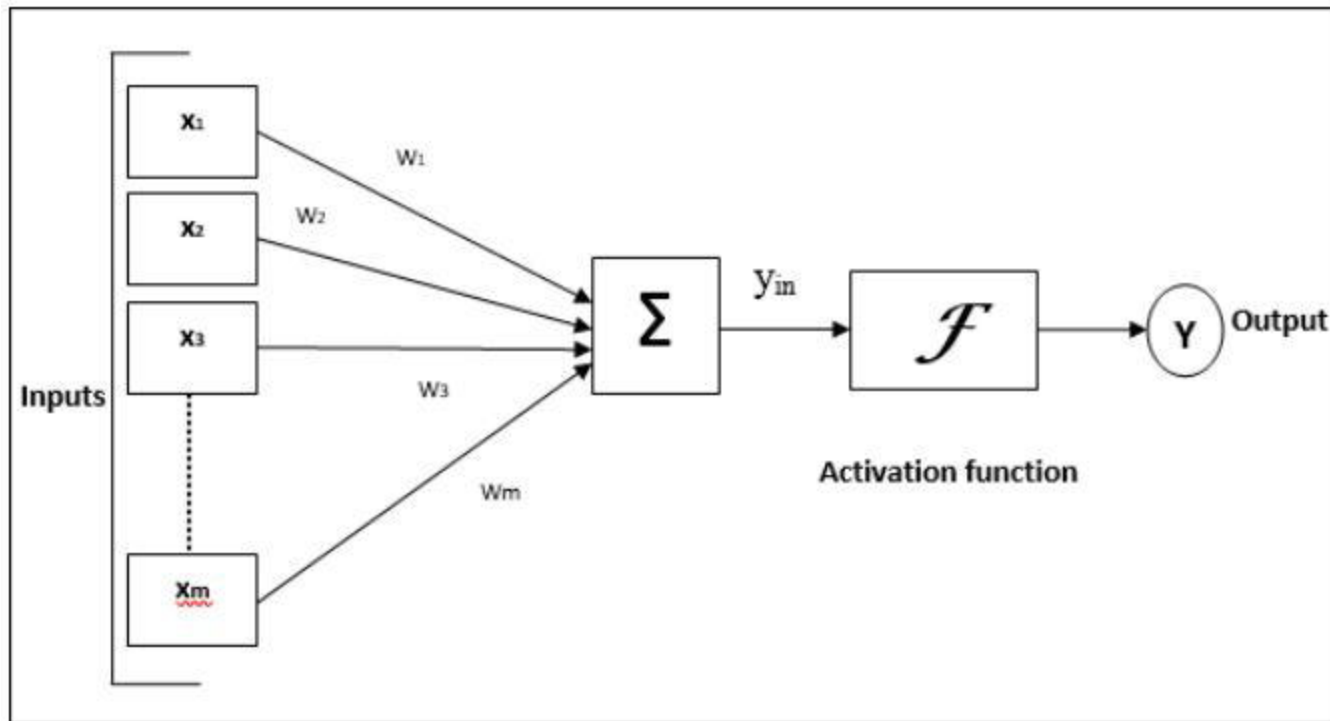
McCulloch-pitts original model consist of

- Inputs: model accepts multiple binary inputs
- Weights: static weights either 0/1
- Summation: All inputs are sum together resulting in total activation value
- Threshold: Threshold value is defined. If summed inputs exceeds the threshold value, neuron fires and produces an o/p 1, otherwise o/p is 0

Alternate Representation of Artificial Neuron



Alternate Representation of Artificial Neuron





Difference Between Biological and Artificial Neuron

Biological neuron	Artificial neuron
dendrites	inputs
synapses	weight or inter connection
axon	output
cell body (Soma)	summation and threshold



Working of Artificial Neuron

An **artificial neuron** is a mathematical function conceived as a model of biological neurons, in a neural network. Artificial neurons are elementary units in an artificial neural network.

The artificial neuron receives one or more inputs and sums them to produce an output by applying some activation function.

For the general model of artificial neural network, the net input can be calculated as follows:

$$y_{in} = (x_1.w_1 + x_2.w_2 + x_3.w_3 \dots x_m.w_m) + \text{bias}$$

i.e., $y_{in} = \sum_i x_i.w_i + \text{bias}$

where, X_i is set of features and W_i is set of weights.



Working of Artificial Neuron

Bias is the information which can impact output without being dependent on any feature. The output can be calculated by applying the activation function over the net input

$$Y=F(y_{in})$$

Each AN has an internal state, which is called an activation signal. Activation signal, which are produced after combining the input signals and activation rule, may be sent to other units.



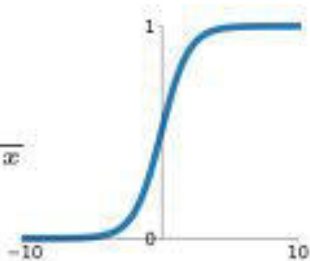
Activation Function:



Activation functions are mathematical equations that determine the output of a neural network's node. They introduce non-linear properties to the network, enabling it to learn complex patterns. Common activation functions include ReLU (Rectified Linear Unit), Sigmoid, and Tanh

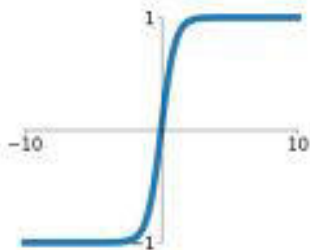
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



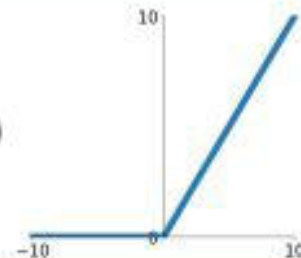
Tanh

$$\tanh(x)$$



ReLU

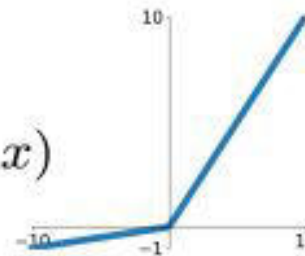
$$\max(0, x)$$



and many
more...

Leaky ReLU

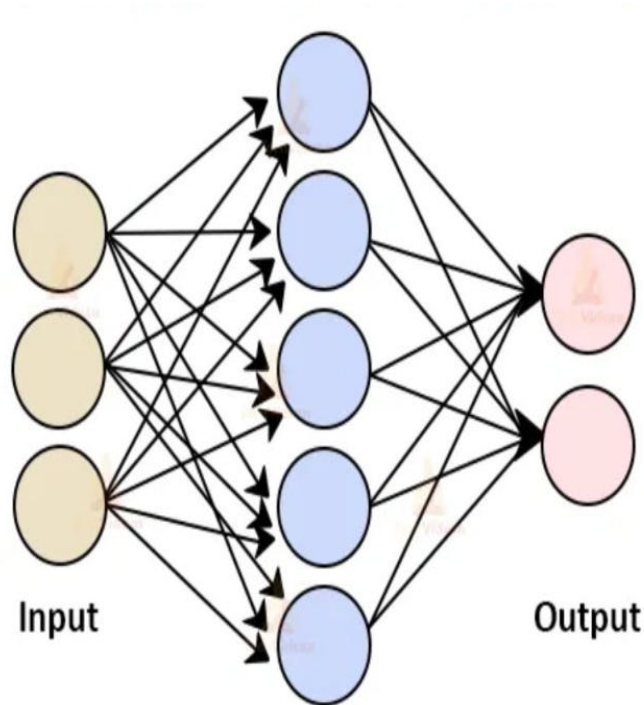
$$\max(0.1x, x)$$



Artificial Neural Network (ANN)

Consist of:

- Input Layer
- Single Hidden Layer
- Output Layer





Types of ANN:



Feed Forward Network

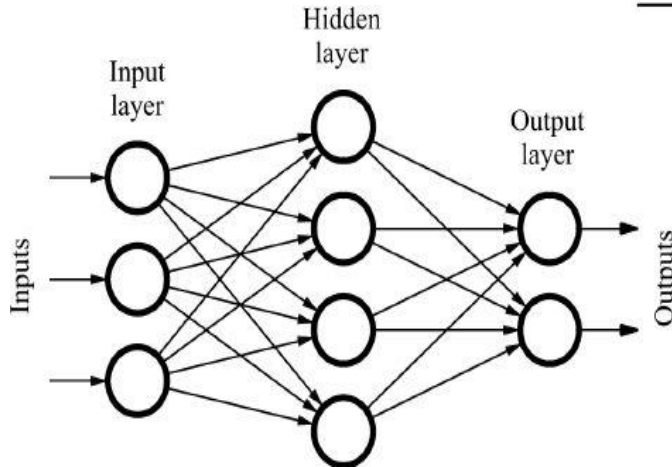
Feedback Network

Recurrent Neural Network

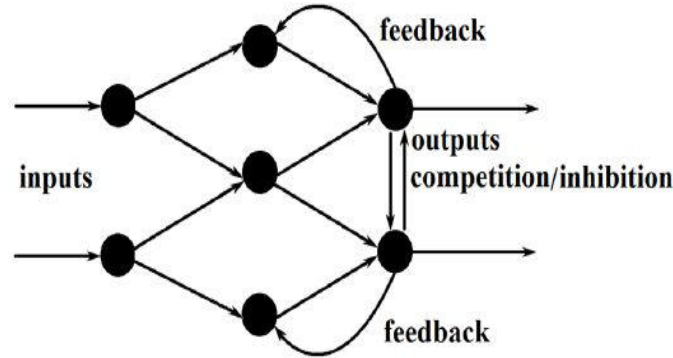


1. Feedforward Neural Network:

- Data flows in one direction-from input layer to the output layer
- Fully connected neurons
- Digit recognition, basic image classification



2. Feedback Network



- Neuron can send information back to previous layers.
- It influence it's own future state by sending output back to earlier layers
- Used in control systems and adaptive systems

3. Recurrent Neural Network:

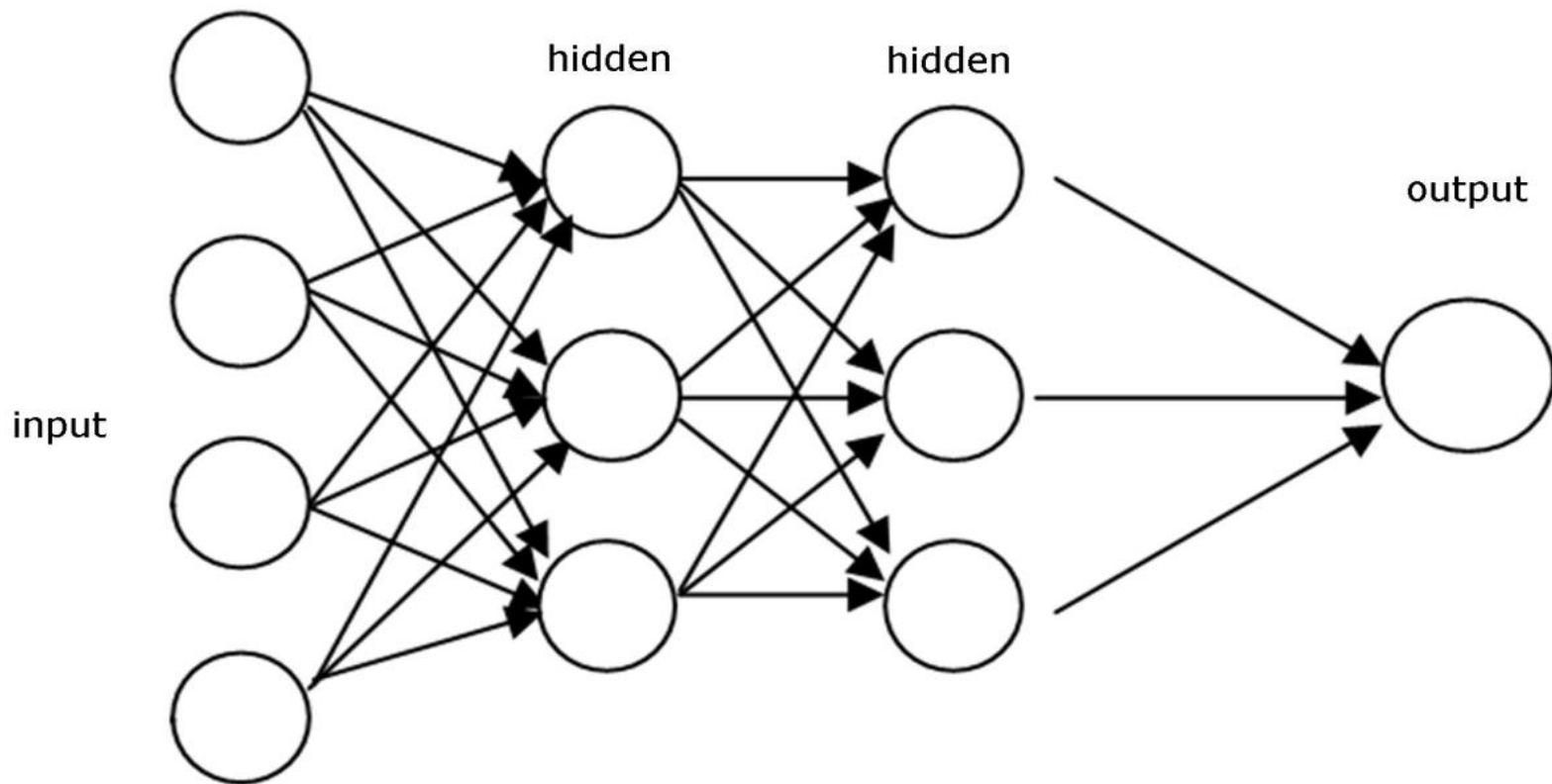
- Each neuron can send outputs back to itself or other neurons in previous layers, creating feedback loop.
- It can capture dependencies in sequence
- Used in NLP(language modelling, text generation), speech recognition and time series analysis



Types of Deep Learning Model:

- Feed Forward Neural Network (FNN)
- Convolutional Neural Network (CNN)
- Recurrent Neural Network (RNN)
- Generative Adversarial Network (GAN)
- Autoencoders
- Transformers
- Graph Neural Network (GNN)
- Deep Reinforcement Learning Models

Feedforward Neural Network





Feedforward Neural Network: Architecture



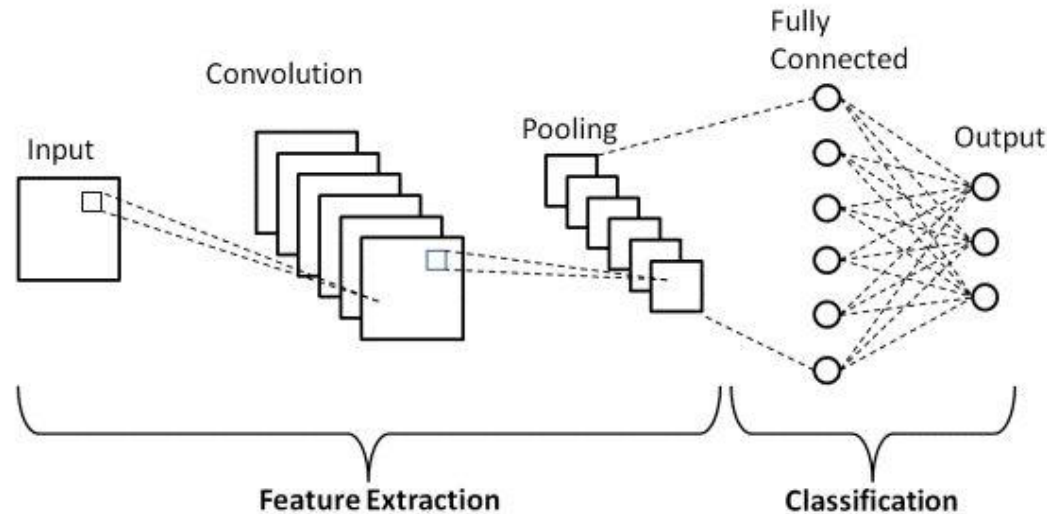
- The architecture of a feedforward neural network consists of three types of layers.
 - the input layer,
 - hidden layers
 - the output layer.
- Each layer is made up of units known as **neurons**, and the layers are interconnected by weights.
- **Input Layer:** This layer consists of neurons that receive inputs and pass them on to the next layer. The number of neurons in the input layer is determined by the dimensions of the input data.
- **Hidden Layers:** These layers are not exposed to the input or output and can be considered as the computational engine of the neural network. Each hidden layer's neurons take the weighted sum of the outputs from the previous layer, apply an **activation function**, and pass the result to the next layer. The network can have zero or more hidden layers.
- **Output Layer:** The final layer that produces the output for the given inputs. The number of neurons in the output layer depends on the number of possible outputs the network is designed to produce.
- Each neuron in one layer is connected to every neuron in the next layer, making this a fully connected network. The strength of the connection between neurons is represented by weights, and learning in a neural network involves updating these weights based on the error of the output



Feedforward Neural Network: How works?

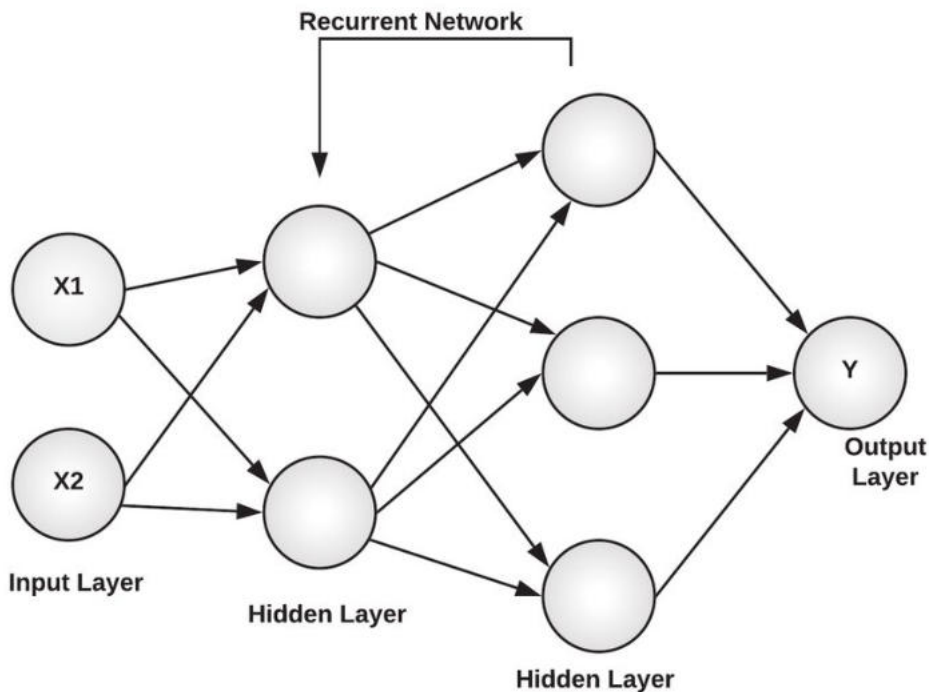
- The working of a feedforward neural network involves two phases: the feedforward phase and the **backpropagation** phase.
- **Feedforward Phase:** In this phase, the input data is fed into the network, and it propagates forward through the network. At each hidden layer, the weighted sum of the inputs is calculated and passed through an activation function, which introduces non-linearity into the model. This process continues until the output layer is reached, and a prediction is made.
- **Backpropagation Phase:** Once a prediction is made, the error (difference between the predicted output and the actual output) is calculated. This error is then propagated back through the network, and the weights are adjusted to minimize this error. The process of adjusting weights is typically done using a gradient descent optimization algorithm.

Convolutional Neural Network (CNN)



- CNNs are a type of neural network that are well-suited for image classification tasks.
- CNNs are composed of a series of convolutional layers, and pooling layers.
- Convolutional layers extract features (called as feature maps) from images by applying a set of filters to an input image.
- Pooling layer is used to reduce the dimensionality of the output(width and height) from the convolutional layer by downsampling the feature maps.
- The output of the convolutional layers is flattened into a one-dimensional vector, which is then passed to the fully connected layers

Recurrent Neural Network (RNN):



- Stores information from previous computations and passes it next layer. i.e. in a sequential manner.

- The output of the previous timestep is used as input for the current timestep along with the new input.

- This creates a relationship between the elements in the sequence, allowing the RNN to learn dependencies between them.
- This makes RNNs well-suited for tasks such as speech.

- RNNs can remember important things about the input they received, which allows them to be very precise in predicting what's coming next. This is why they're the preferred

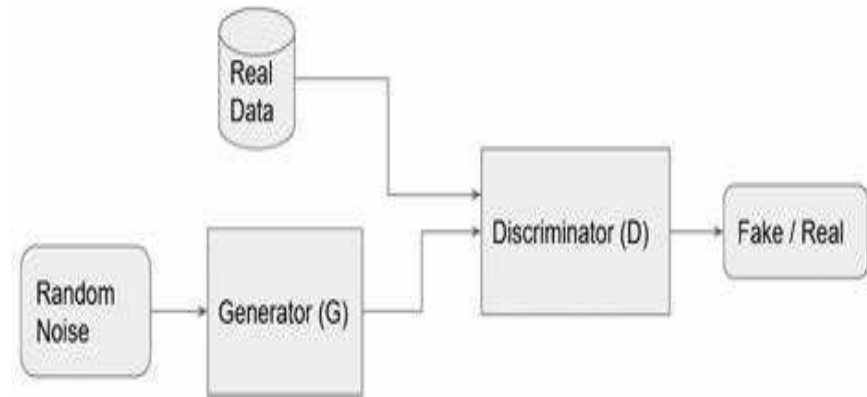
Fundamental algorithm for sequential data like time series, speech, text, financial data, audio, video.



Generative Adversarial Network (GAN)

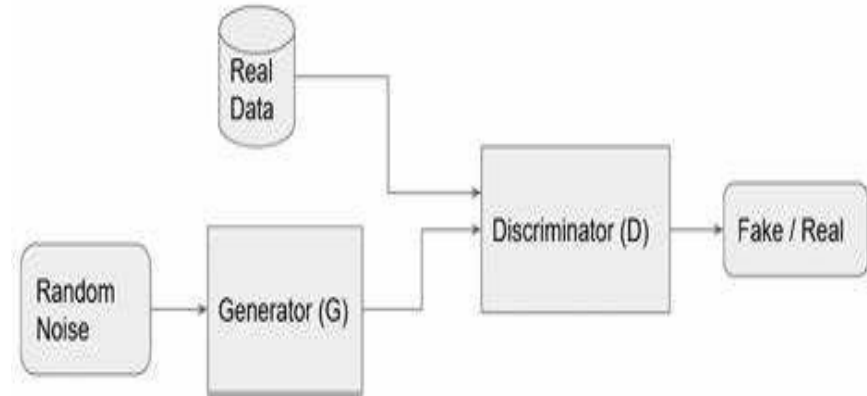


- **Imagine this:**
- You have two students: a **forger** and a **detective**.
- The **forger** tries to create fake paintings that look like real ones.
- The **detective** tries to spot the fake paintings from the real ones.
- **That's basically how a GAN works:**
- **Generator:** Like the forger, it's a neural network that creates fake data (images, text, etc.).
- **Discriminator:** Like the detective, it's a neural network that tries to tell the difference between real data and the fake data created by the generator.
- **How they learn:**
- **The generator gets better:** It learns to create more and more realistic fake data, so the detective has a harder time spotting them.



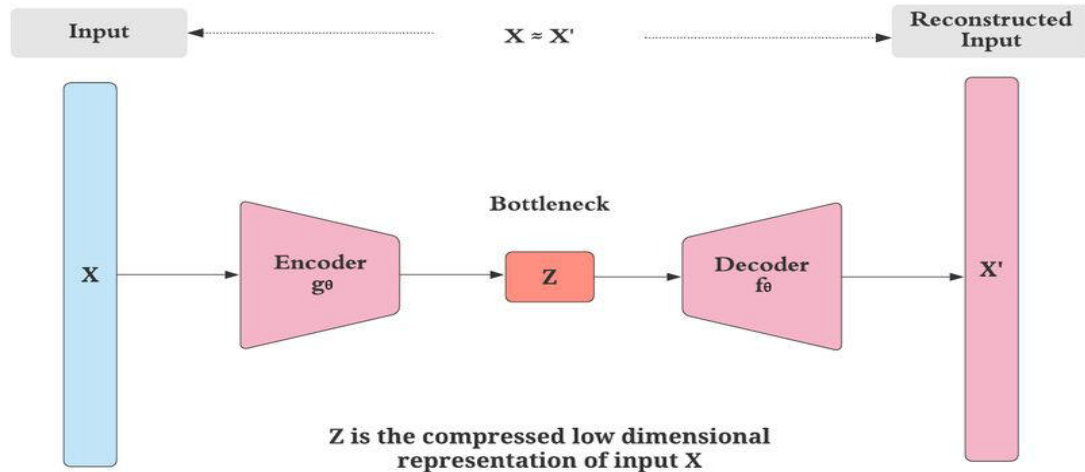
Generative Adversarial Network (GAN)

- **The detective gets better:** It learns to be a better detective, finding even the most convincing fake data.
- **This goes on and on:**
- The generator and detective keep competing, and in the process, both get better and better.
- Eventually, the generator becomes so good at creating fake data that the detective can't tell the difference anymore!
- **Why are GANs useful?**
- **Creating realistic images:** GANs can generate images of people, objects, or scenes that look incredibly real.
- **Improving image quality:** They can make blurry images sharper or add details to low-resolution images.
- **Generating new data:** They can create new examples of data, which can be useful for training other machine learning models.





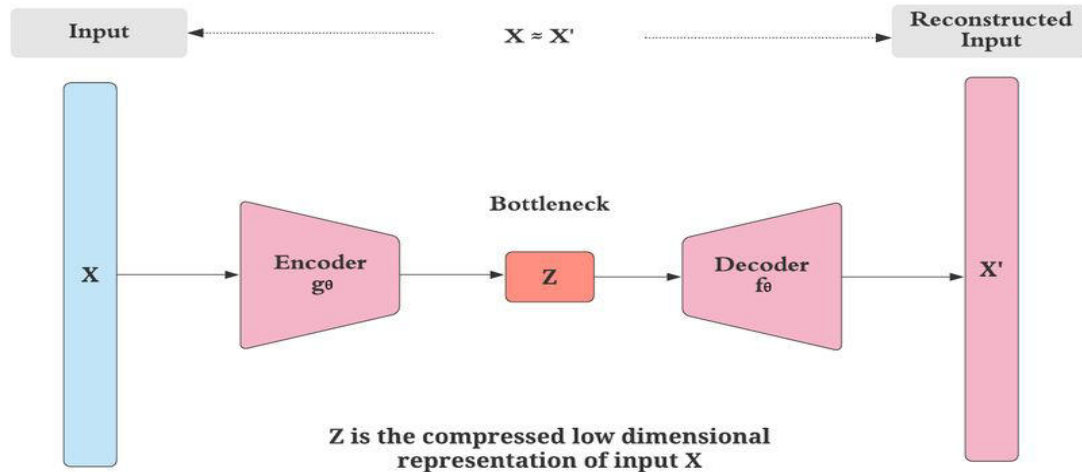
Autoencoders



- It's a type of neural network that learns to compress data (like your cat photo) into a smaller, simpler form (like a zip file).
- It can then uncompress that smaller form back into something that looks a lot like the original data.

How it works:

1. **Encoder:** This part of the network takes your cat photo and squishes it down into a smaller, more compact representation. Think of it like creating a summary of the photo.
2. **Code:** This is the compressed version of your photo. It contains the most important information needed to recreate the image.
3. **Decoder:** This part of the network takes the compressed code and tries to reconstruct the original cat photo as best as it can.



Why is this useful?

- **Saving space:** Autoencoders can compress data, making it easier to store or transmit.
- **Finding important features:** The compressed code contains the most important information about the data, which can be useful for other tasks like image recognition.
- **Removing noise:** Autoencoders can be used to clean up noisy data, like removing static from a recording.



Transformers:

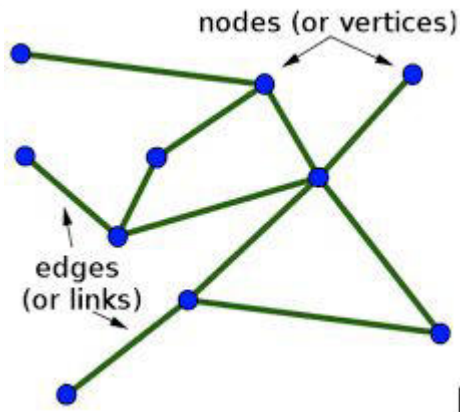
What they do: These are a type of neural network that has revolutionized natural language processing (NLP). They excel at understanding context and relationships in sequential data, like text.

How they work: They use a mechanism called "attention" to weigh the importance of different words in a sentence when processing it. This allows them to capture long-range dependencies and understand the meaning of a sentence much better than previous models.

Why they're important: Transformers power many of the language-based AI applications we use today, such as:

- Machine translation (Google Translate)
- Chatbots
- Text summarization
- Question answering

Graph Neural Network(GNN)



Imagine this:

- You have a map of a city, with roads connecting different locations.
- You want to understand how traffic flows through the city, or which locations are most important.

GNNs are great for this kind of problem:

- They are a type of neural network designed to work with data that has a graph structure, like your city map.
- They can learn from the connections between different parts of the graph, and use that information to make predictions or understand the data better.

How it works:

1. **Nodes:** In your city map, the locations (like intersections or landmarks) are the nodes of the graph.
2. **Edges:** The roads connecting the locations are the edges of the graph.
3. **Features:** Each node might have some information associated with it, like the number of people living there or the type of businesses located there.
4. **Message passing:** GNNs work by passing messages between the nodes of the graph. Each node gathers information from its neighbors, and uses that information to update its own representation.
5. **Learning:** By repeating this process, the GNN learns to understand the relationships between the nodes and edges in the graph, and can use that information to solve problems.



Graph Neural Network(GNN)

Why are they useful?

- **Understanding relationships:** GNNs can help us understand complex relationships between different entities, like social networks, transportation networks, or even molecules in a chemical compound.
- **Making predictions:** They can be used to predict things like traffic flow, customer behavior, or the properties of a molecule.
- **Solving real-world problems:** GNNs are being used in a wide range of applications, including:
 - Social network analysis
 - Recommendation systems
 - Drug discovery
 - Traffic prediction

Think of it like this:

- A GNN is like a group of people working together to solve a puzzle.
- Each person has some information about the puzzle, and they share that information with their neighbors.
- By working together, they can combine their knowledge and solve the puzzle more effectively.

Key points to remember:

- GNNs are designed to work with data that has a graph structure.
- They learn by passing messages between the nodes of the graph.
- They can be used to understand relationships, make predictions, and solve real-world problems.



Deep Reinforcement Learning Model

- **Reinforcement Learning (RL):** Imagine training a dog. You give it a treat when it does something good (positive reward) and maybe a "no" when it does something bad (negative reward). Over time, the dog learns to do more of the good things to get more treats. RL is similar: an agent learns to make decisions by interacting with an environment and getting rewards or penalties.
- **Deep Learning (DL):** You've already learned about neural networks. They're great at recognizing patterns in data, like images or text.

Deep Reinforcement Learning combines these two:

- It uses deep neural networks to learn how to make good decisions in an environment.
- The neural network acts as the "brain" of the agent, figuring out the best actions to take based on the rewards it receives.

Why is this powerful?

- **Learning complex tasks:** DRL can handle problems with lots of possible actions and complex environments, like playing video games or driving a car.
- **Learning from raw data:** DRL can learn directly from raw sensory input, like images from a camera, without needing someone to manually define the important features.

Here's how it works:

1. **Agent:** This is the learner, like your dog or a robot.
2. **Environment:** This is the world the agent interacts with, like a game or a physical space.
3. **State:** This is the current situation the agent is in, like the dog seeing a ball or the robot facing an obstacle.
4. **Action:** This is what the agent does, like the dog fetching the ball or the robot moving forward.
5. **Reward:** This is feedback the agent gets, like a treat for the dog or a score in the game.



Deep Reinforcement Learning Model

The DRL model learns through trial and error:

- The agent takes actions and observes the rewards it gets.
- It adjusts its neural network to make better decisions in the future, aiming to maximize the total reward it receives.

Some popular DRL models:

- **Q-learning:** Learns a "Q-value" for each action in each state, representing how good that action is.
- **Deep Q-Network (DQN):** A type of Q-learning that uses a deep neural network to estimate Q-values.
- **Policy Gradient:** Directly learns a "policy" that maps states to actions, without estimating Q-values.
- **Actor-Critic:** Combines Q-learning and policy gradient methods, with an "actor" that chooses actions and a "critic" that evaluates them.

Applications of DRL:

- **Games:** Playing Atari games, Go, and even complex video games like Dota 2.
- **Robotics:** Controlling robots to perform tasks in the real world.
- **Self-driving cars:** Learning to navigate roads and make driving decisions.
- **Recommendation systems:** Suggesting products or content to users.

Key points to remember:

- DRL combines reinforcement learning and deep learning.
- It uses neural networks to learn how to make decisions in an environment.
- It learns through trial and error, maximizing rewards.
- It has many applications in games, robotics, and other fields.



Application of Deep Learning:

There are various interesting applications for Deep Learning that made impossible things before a decade into reality. Some of them are:

1. Color restoration, where a given image in grayscale is automatically turned into a colored one.
2. Recognizing handwritten message.
3. Adding sound to a silent video that matches with the scene taking place.
4. Self-driving cars
5. Computer Vision: for applications like vehicle number plate identification and facial recognition.
6. Information Retrieval: for applications like search engines, both text search, and image search.
7. Marketing: for applications like automated email marketing, target identification
8. Medical Diagnosis: for applications like cancer identification, anomaly detection
9. Natural Language Processing: for applications like sentiment analysis, photo tagging
10. Online Advertising, etc